



Università degli Studi di Napoli Federico II
Scuola Politecnica e delle Scienze di Base

Dipartimento di Ingegneria Elettrica e Tecnologie dell'Informazione

Corso di Laurea Triennale in Ingegneria dell'Automazione

Controllo cinematico di un robot manipolatore tramite algoritmo di cinematica inversa

Relatore
Prof. Luigi Villani

Candidato
Emanuele Tieri
N39/001359

Anno Accademico 2023–2024

Università degli Studi di Napoli Federico II
Scuola Politecnica e delle Scienze di Base
Dipartimento di Ingegneria Elettrica e Tecnologie dell'Informazione
Corso di Laurea Triennale in Ingegneria dell'Automazione

**CONTROLLO CINEMATICO DI UN ROBOT
MANIPOLATORE TRAMITE ALGORITMO DI
CINEMATICA INVERSA**

Relatore
Prof. Luigi VILLANI

Candidato
Emanuele TIERI
N39/001359

Anno Accademico 2023–2024

Indice

1. Introduzione	- 5 -
1.1. Struttura della tesi	- 6 -
2. Cinematica.....	- 7 -
2.1. Robot manipolatori.....	- 7 -
2.2. Kinova Jaco2 7DOF	- 9 -
2.3. Posa di un corpo rigido.....	- 11 -
2.4. Trasformazioni omogenee	- 13 -
2.5. Cinematica diretta.....	- 15 -
2.5.1. Catena cinematica aperta	- 16 -
2.5.2. Convenzione di Denavit-Hartenberg (DH)	- 17 -
2.5.3. Spazio di lavoro	- 19 -
3. Inversione della cinematica differenziale e ridondanza	- 20 -
3.1. Cinematica differenziale e Jacobiano.....	- 20 -
3.2. Inversione della cinematica differenziale.....	- 21 -
3.3. Manipolatori ridondanti.....	- 22 -
3.4. Algoritmi per l'inversione cinematica: il CLIK	- 24 -
4. Implementazione dell'algoritmo e simulazione	- 26 -

Bibliografia

Sitografia

Ringraziamenti

1. Introduzione

La robotica rappresenta uno dei campi più affascinanti e in continua evoluzione dell'ingegneria e della tecnologia moderna. Nata dalla fusione di discipline quali l'elettronica, la meccanica, l'informatica e l'intelligenza artificiale, la robotica ha rivoluzionato il modo in cui interagiamo con il mondo che ci circonda, con l'obiettivo di creare macchine intelligenti e autonome in grado di svolgere compiti complessi, migliorando l'efficienza, la precisione e la sicurezza delle operazioni umane in una vasta gamma di settori. Basti pensare che i primi tentativi di creare "robot" risalgono all'antichità, con i meccanismi ingegnosi ideati da inventori e studiosi come Archimede e Leonardo da Vinci. Tuttavia, è nel corso del XX secolo che la robotica ha fatto passi da gigante, con lo sviluppo dei primi robot industriali nei laboratori di ricerca e nelle fabbriche automobilistiche, fino ad arrivare alla creazione dei robot manipolatori odierni.

È infatti obiettivo di tale tesi concentrarsi sullo studio e lo sviluppo di algoritmi avanzati di controllo cinematico per il braccio robotico Kinova Jaco2 7DOF, un manipolatore con ridondanza a 7 gradi di libertà (7DOF) che offre una notevole flessibilità e versatilità nelle operazioni di manipolazione.

Tuttavia, la presenza di ridondanza introduce complessità nel controllo del movimento, poiché esistono molteplici configurazioni delle articolazioni che possono raggiungere la stessa posizione nello spazio operativo. Questa ridondanza richiede l'implementazione di algoritmi specifici per determinare la configurazione ottimale delle articolazioni in base agli obiettivi proposti, tenendo conto dei vincoli cinematici, delle limitazioni fisiche e delle specifiche del compito da eseguire.

Lo studio e la modellazione del manipolatore sono stati eseguiti tramite l'uso di Matlab, usando il Robotics Toolbox di Peter Corke, visualizzando poi il movimento del braccio sul software CoppeliaSim.

1.1. Struttura della tesi

Prima di cimentarsi nell'implementazione dell'algoritmo, si affronteranno nei Capitoli 2 e 3 i fondamenti teorici che sono alla base del controllo del robot manipolatore. In particolare, nel paragrafo 2.1 vi sarà una panoramica sui tipi di manipolatori robotici, con un'attenzione al Kinova Jaco2 7DOF nel 2.2; nel paragrafo 2.3 ci si sofferma sulla posa di un corpo rigido; quindi nel 2.4 sulle trasformazioni omogenee; nel 2.5 e nei suoi sottoparagrafi (2.5.1, 2.5.2, 2.5.3) si parlerà di cinematica diretta e della convenzione di Denavit-Hartenberg, del concetto di catena aperta e dello spazio di lavoro.

Il Capitolo 3 sarà dedicato alla cinematica inversa e alla ridondanza; nel dettaglio si partirà dal paragrafo 3.1 con un'analisi sulla cinematica differenziale e sullo Jacobiano; si proseguirà poi nel paragrafo 3.2 con l'inversione cinematica differenziale e i problemi che ne derivano; nel 3.3 si capirà cosa differenzia un manipolatore ridondante dagli altri manipolatori non ridondanti (e quindi si parlerà del funzionale di costo), per poi terminare con il paragrafo 3.4 con l'illustrazione dell'algoritmo CLIK (Closed Loop Inverse Kinematics) tramite l'uso della pseudo-inversa dello Jacobiano.

Nel Capitolo 4 sarà centrale l'implementazione dell'algoritmo di Matlab con la sua simulazione in CoppeliaSim: verrà data una spiegazione dell'algoritmo e delle funzioni utilizzate, nonché un'illustrazione dei risultati e delle simulazioni (vi sarà esposto il codice per intero alla fine del capitolo).

2. Cinematica

2.1. Robot manipolatori

I robot manipolatori sono macchine programmabili progettate per movimentare materiali, pezzi o strumenti. Essi sono formati da dei corpi rigidi, chiamati *bracci*, collegati tra loro tramite *giunti*, i quali permettono un moto relativo tra due bracci consecutivi.

I giunti possono essere *rotoidali* o *prismatici*: i primi consentono il movimento di rotazione attorno ad un asse (tra questi giunti a snodo sferico e giunti cardani); i secondi permettono il movimento lungo una linea retta (ad esempio come un carrello che scorre lungo una guida).

Inoltre, nel manipolatore individuiamo il *polso* che è la parte che consente il movimento dell'organo terminale rispetto all'ultima sezione della catena cinematica. L'insieme dei collegamenti tra giunti che compongono il robot prende il nome di *struttura portante*, o struttura principale del manipolatore, al termine del quale riconosciamo *l'organo terminale*, o end-effector, che è l'elemento finale del manipolatore che interagisce direttamente con l'ambiente. Esso può essere una pinza, un utensile, una ventosa o qualsiasi altro dispositivo in grado di afferrare, sollevare o manipolare gli oggetti.

La sequenza di collegamenti e giunti che connettono la base del robot all'organo terminale si definisce *catena cinematica*. Essa definisce il percorso che l'organo terminale può seguire nello spazio. Riconosciamo:

- *catena cinematica aperta*: una catena cinematica in cui l'organo terminale non è collegato al manipolatore da altri collegamenti, in cui ogni giunto prismatico o rotoideale conferisce un solo grado di libertà.
- *catena cinematica chiusa*: una catena cinematica in cui l'organo terminale è collegato al manipolatore attraverso altri collegamenti, formando un loop.

I *gradi di libertà* sono prerogative dei giunti; essi, infatti, devono essere distribuiti opportunamente lungo tutta la struttura del braccio robotico, in modo da essere in numero sufficiente per far svolgere al manipolatore un dato compito. Generalmente i gradi di libertà richiesti per posizionare ed orientare un oggetto nello *spazio di lavoro* (esso rappresenta

l'ambiente in cui l'organo terminale può accedere entro i limiti fisici del robot e dei *fine corsa* dei giunti) sono 6, tre per il posizionamento e tre per l'orientamento di quest'ultimo nello spazio, rispetto ad una terna di coordinate di riferimento fissata.

Se il numero di gradi di libertà supera in numero quello dei vincoli dei giunti si parla di *ridondanza*; i bracci ridondanti sono manipolatori che hanno più giunti di quelli necessari per raggiungere una determinata posizione nello spazio. Questa ridondanza offre vantaggi come la capacità di evitare ostacoli, migliorare la flessibilità operativa e ottimizzare le prestazioni complessive del manipolatore, offrendo quindi una molteplice scelta della configurazione che i giunti posso assumere per arrivare ad una stessa posizione dello spazio. Si richiede, come vedremo nel Cap. 3, quindi l'implementazione di algoritmi specifici per riuscire ad individuare la configurazione ottimali, in termini di risparmio energetico ed aumento di prestazioni.

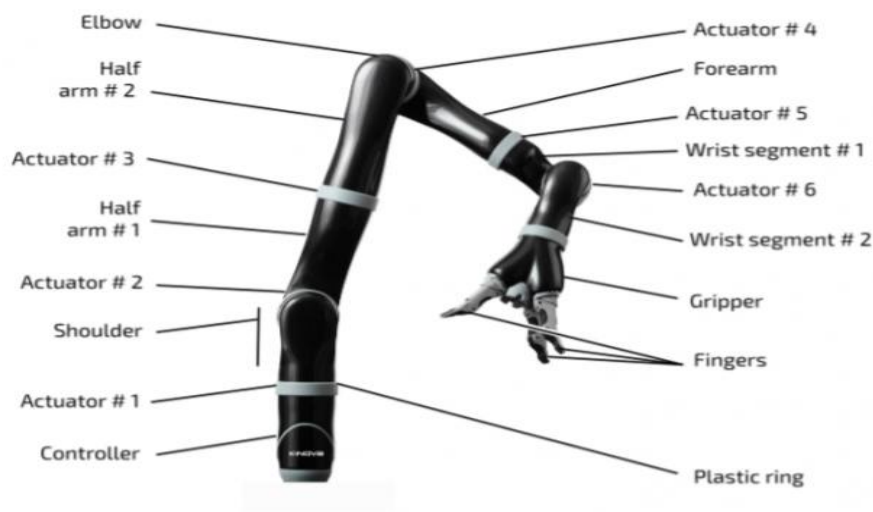


Figura 1 – Componenti di un Kinova Jaco2 con 7DOF

2.2. Kinova Jaco2 7DOF

Il Kinova Jaco2 a 7 DOF è una versione avanzata del braccio robotico Jaco2 (a 6 gradi di libertà), con un grado di libertà aggiuntivo, portando il totale a sette, permettendo al manipolatore di raggiungere posizioni e orientamenti precisi nell'ambiente circostante. Esso è un robot leggero (5,98 kg con pinza a due dita e 6,18 kg con pinza a 3 dita) progettato con un'architettura modulare che facilita l'integrazione di end-effector personalizzati e altri accessori come pinze, ventose, sensori e strumenti specializzati. Il dispositivo è fornito di un sistema di comando che permette all'utilizzatore di muovere il robot per svolgere una varietà di compiti, quali il sollevamento, la presa e la manipolazione di oggetti, raggiungendo una velocità massima di 20 cm/s.

Grazie alla sua versatilità, infatti, il Kinova Jaco2 7DOF trova applicazioni in una vasta gamma di settori, tra cui l'assistenza agli anziani e ai disabili, l'automazione industriale, la medicina e la riabilitazione.



Figura 2 - Kinova Jaco2 7DOF

Il robot è in grado di compiere diversi movimenti che possono essere suddivisi in quattro modalità di controllo: modalità traslazione, modalità polso, modalità bevuta e infine modalità dita.

Nella **modalità traslazione**, l'utente gestisce la posizione della mano nello spazio mantenendo costantemente il suo orientamento parallelo al telaio del sedile della carrozzina. I movimenti consentiti sono spostamenti laterali, avanti e indietro, e su e giù della mano.

In **modalità polso**, l'utente controlla la posizione del braccio rispetto a un punto centrale della mano, noto come punto di riferimento, che rimane stabile o si muove solo leggermente. I movimenti laterali si riferiscono alla rotazione circolare del polso intorno al punto di

riferimento, mentre i movimenti verticali si riferiscono alla rotazione circolare superiore o inferiore del polso. La rotazione del polso indica il movimento circolare della mano su sé stessa. La **modalità bevuta** è utilizzata esclusivamente per la rotazione del polso. In questa modalità, il punto di riferimento è posizionato al di fuori dell'asse di altezza e lunghezza per creare una rotazione compensata quando l'utente beve da un bicchiere o una bottiglia senza cannuccia. Nella **modalità dita**, l'utente controlla l'apertura e la chiusura di due o tre dita del manipolatore.

Jaco2 è progettato per essere controllato utilizzando il joystick della carrozzina elettrica. In alternativa, ci sono altre opzioni di controllo disponibili, come il joystick Kinova a tre assi (sinistra/destra, avanti/dietro, rotazione), qui rappresentato in figura:

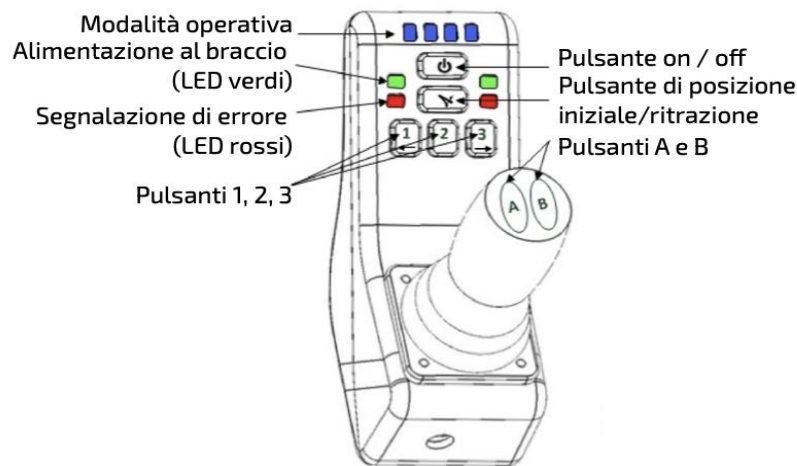


Figura 3 - Joystick Kinova

Joint	Min (rad)	Max (rad)
1	$-\infty$	∞
2	0.82	5.46
3	$-\infty$	∞
4	0.52	5.76
5	$-\infty$	∞
6	1.13	5.14
7	$-\infty$	∞

Tabella 1 – Limiti di giunto dello Jaco2

2.3. Posa di un corpo rigido

Per individuare un *corpo rigido* nello spazio si fa riferimento alla sua *posizione* e al suo *orientamento*. La posizione è determinata dal suo punto di riferimento, che è di solito il centro di massa o un punto specifico sul corpo. Questo punto è espresso tramite le sue coordinate spaziali, che possono essere definite rispetto a un sistema di riferimento fisso; mentre l'orientazione di un corpo rigido si riferisce alla direzione in cui è rivolto. Può essere definita tramite angoli di rotazione attorno agli assi dello spazio o tramite una matrice di rotazione che descrive la trasformazione lineare dal sistema di riferimento del corpo a un sistema di riferimento fisso. Tutto ciò prende il nome di *posa* del corpo rigido.

Sia O -xyz la terna ortonormale di coordinate di riferimento e indichiamo con x, y, z i tre versori degli assi della terna. Indicati con o'_x, o'_y, o'_z le componenti del vettore $o' \in R^3$ lungo gli assi della terna per individuare la posizione di un punto O' di un corpo rigido nello spazio rispetto alla terna O -xyz si utilizza la seguente espressione:

$$o' = o'_x x + o'_y y + o'_z z \quad (2.1)$$

Si osserva cioè che la posizione di O' è descritta dal vettore di dimensione (3×1) $o' = [o'_x, o'_y, o'_z]'$, dove tale vettore viene definito *applicato* in quanto è specificato, oltre alla direzione e al modulo, il suo punto di applicazione.

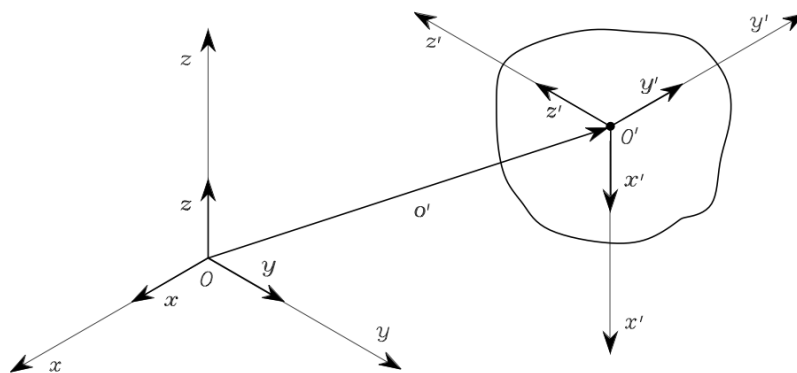


Figura 4 - Posa di un corpo rigido

Per definire invece l'orientamento del corpo rigido, si introduce una terna ortonormale solidare al corpo e si esprimono i versori di tale terna rispetto alla terna di riferimento.

Indichiamo quindi tale terna con $O'-x'y'z'$, con origine in O' , e i rispettivi versori x', y', z' , che sono espressi rispetto a $O-xyz$ dal seguente sistema di equazioni:

$$\begin{aligned}x' &= x'_x x + x'_y y + x'_z z \\y' &= y'_x x + y'_y y + y'_z z \\z' &= z'_x x + z'_y y + z'_z z\end{aligned}\tag{2.2}$$

Tali versori possono essere espressi in maniera più compatta tramite la *matrice di rotazione* (3x3):

$$R = \begin{bmatrix} x' & y' & z' \end{bmatrix} = \begin{bmatrix} x'_x & y'_x & z'_x \\ x'_y & y'_y & z'_y \\ x'_z & y'_z & z'_z \end{bmatrix} = \begin{bmatrix} x'^T x & y'^T x & z'^T x \\ x'^T y & y'^T y & z'^T y \\ x'^T z & y'^T z & z'^T z \end{bmatrix}\tag{2.3}$$

Notiamo che

$$x'^T y' = 0 \quad y'^T z' = 0 \quad z'^T x' = 0$$

cioè che sono ortogonali tra di loro dato che rappresentano i versori di una terna ortonormale, ed hanno modulo unitario:

$$x'^T x' = 1 \quad y'^T y' = 1 \quad z'^T z' = 1$$

La matrice R quindi è ortonormale e vale la seguente proprietà:

$$R^T R = I_3\tag{2.4}$$

con I_3 matrice identità (3x3).

Moltiplicando infine ambo i membri per R^{-1} si ottiene

$$R^T = R^{-1}\tag{2.5}$$

La matrice R così definita si dice che appartiene al *gruppo speciale ortonormale* $SO(m)$ delle *matrici reali* ($m \times m$) con colonne ortonormali e $\det(R) = 1$.

2.4. Trasformazioni omogenee

Una trasformazione omogenea è una rappresentazione matriciale delle trasformazioni geometriche nello spazio tridimensionale, che include traslazioni, rotazioni e scalamenti. Esse sono uno strumento fondamentale nella descrizione della cinematica dei robot e dei sistemi meccanici in generale. Queste trasformazioni sono descritte tramite una matrice 4x4, chiamata *matrice di trasformazione omogenea*.

Si consideri un punto P nello spazio, la cui posizione rispetto alla terna $O_0 - x_0y_0z_0$ è individuata dal vettore p_0 . Consideriamo poi un'altra terna $O_1 - x_1y_1z_1$, dove o_1^0 è il vettore che individua l'origine della terna 1, O_1 , rispetto a O_0 , R_1^0 la matrice di rotazione della terna 1 rispetto alla terna 0; sia p^1 il vettore coordinate di P rispetto alla terna 1, ricaviamo la posizione di P rispetto a O_0 come:

$$p^0 = o_1^0 + R_1^0 p^1 \quad (2.6)$$

tale relazione esprime la trasformazione di coordinate (per la traslazione e la rotazione) di un vettore applicato da una terna all'altra.

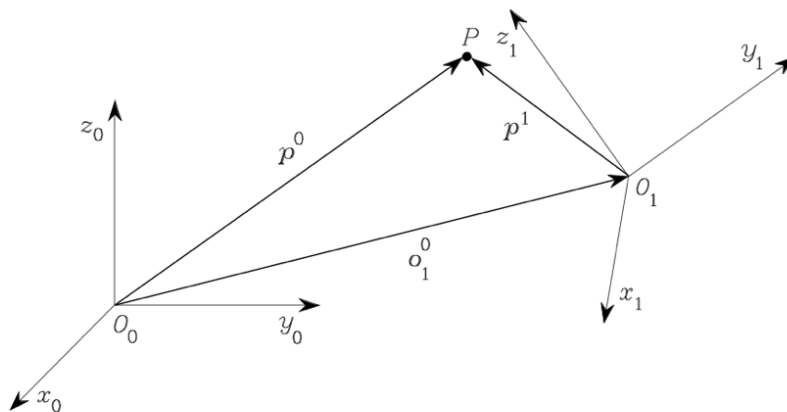


Figura 5 - Trasformazione omogenea

Per ricavare la trasformazione inversa, cioè p^1 , moltiplichiamo ambo i membri della (2.6) per R_1^{0T} , e tenendo presente della proprietà di ortogonalità di una matrice di rotazione e che

$$R_i^j = (R_j^i)^{-1} = (R_j^i)^T \quad (2.7)$$

otteniamo

$$p^1 = -R_0^1 o_1^0 + R_0^1 p^0 \quad (2.8)$$

Introduciamo a tal punto la *rappresentazione omogenea* del vettore p , definendo il vettore

$$\hat{p} = \begin{bmatrix} p \\ 1 \end{bmatrix} \quad (2.9)$$

che ci permette di definire la *matrice di trasformazione omogenea* A_1^0

$$A_1^0 = \begin{bmatrix} R_1^0 & o_1^0 \\ 0^T & 1 \end{bmatrix} \quad (2.10)$$

dove la riga inferiore $[0 \ 0 \ 0 \ 1]$ è stata aggiunta per mantenere la dimensione 4x4;

$o_1^0 \in R^3$ e $R_1^0 \in SO(3)$, quindi la matrice A_1^0 appartiene al *gruppo speciale euclidea* $SE(3) = R^3 \times SO(3)$.

Analogamente la matrice A_0^1 , che descrive la trasformazione di coordinate tra la terna 0 e la terna 1, si scrive

$$A_0^1 = \begin{bmatrix} R_0^1 & -R_0^1 o_1^0 \\ 0^T & 1 \end{bmatrix} \quad (2.11)$$

2.5. Cinematica diretta

La cinematica diretta è un concetto fondamentale nella robotica e nella meccanica, che si occupa di determinare la posizione e l'orientamento di un organo terminale (end-effector) o di qualsiasi punto di interesse di un robot, rispetto alla terna base, date le posizioni dei suoi giunti. Ricordiamo che l'insieme dei bracci e dei giunti di un robot manipolatore forma la catena cinematica, in cui un estremo della catena è vincolato alla base e l'altro, l'end-effector, può o meno esserlo.

Si parlerà infatti di catena cinematica aperta se ci si riferisce a una sequenza di collegamenti rigidi congiunti in serie, che collegano il punto di origine al punto di destinazione senza formare anelli chiusi. Questo tipo di catena è comune nei robot seriali, come il braccio robotico industriale.

D'altra parte, una catena cinematica chiusa include almeno un anello chiuso di collegamenti rigidi. Questo tipo di catena è più comune nei robot paralleli, dove più bracci o membri sono collegati in modo tale da formare una struttura chiusa. Le catene cinematiche chiuse possono offrire vantaggi come una maggiore rigidità e capacità di carico, ma possono anche rendere più complessa l'analisi cinematica.

La cinematica diretta, o anche nota come "problema diretto", riguarda il calcolo della posizione e dell'orientamento dell'organo terminale del robot dato lo stato (posizione e orientamento) dei suoi giunti. Questo processo si basa sulla trasformazione omogenea che rappresenta la posa dell'organo terminale rispetto al sistema di riferimento del robot.

In tal senso ha notevole importanza il concetto di *postura* di una catena cinematica, con cui si indica la descrizione della posa di tutti i corpi rigidi che la costituiscono. La postura di un robot, infatti, si riferisce alla sua configurazione geometrica nello spazio, definita dalle posizioni e dagli orientamenti dei suoi giunti. Le *variabili di giunto* sono i parametri utilizzati per descrivere la posizione dei giunti di un robot. Queste variabili possono essere angoli (variabili angolari) per giunti rotoidali o distanze (variabili di traslazione) per giunti prismatici.

Consideriamo una terna di riferimento $O_b - x_b y_b z_b$, la funzione cinematica diretta è espressa mediante la trasformazione omogenea di seguito riportata

$$T_e^b(q) = \begin{bmatrix} n_e^b(q) & s_e^b(q) & a_e^b(q) & p_e^b(q) \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.12)$$

dove q ($n \times 1$) è il vettore delle variabili di giunto, n_e, s_e, a_e , sono i versori della terna solidare all'organo terminale, e p_e è il vettore posizione dell'origine della terna solidare all'organo terminale del manipolatore rispetto all'origine della terna $O_b - x_b y_b z_b$, che viene definita *terna base*; la terna solidare all'organo terminale è definita *terna utensile*.

2.5.1. Catena cinematica aperta

Consideriamo un manipolatore a catena aperta costituito da una serie di bracci connessi tramite giunti. Questa catena è composta da $n+1$ bracci, dove il primo braccio è fissato a terra. Ogni giunto offre un singolo grado di libertà alla struttura, corrispondente alla variabile di giunto. Poiché ogni giunto collega esattamente due bracci consecutivi, è ragionevole affrontare separatamente il problema della descrizione dei legami cinematici tra bracci adiacenti. Successivamente, possiamo risolvere in modo ricorsivo il problema della descrizione complessiva della cinematica del manipolatore, considerando le relazioni tra le trasformazioni omogenee di ciascun braccio lungo la catena.

Definiamo allora una terna di coordinate solidale ad ogni braccio (dal braccio 0 a n), in modo tale da poter definire la trasformazione di coordinate complessiva $T_n^0(q)$ che esprime posizione ed orientamento della terna n rispetto alla terna 0, ottenuta tramite prodotti di matrici di trasformazione omogenea $A_i^{i-1}(q_i)$:

$$T_n^0(q) = A_1^0(q_1) A_2^1(q_2) \dots A_n^{n-1}(q_n) \quad (2.13)$$

In definitiva la trasformazione di coordinate che descrive posizione ed orientamento della terna utensile rispetto alla terna base si scrive:

$$T_e^b(q) = T_0^b T_n^0(q) T_e^n \quad (2.14)$$

con T_0^b e T_e^n trasformazioni omogenee che descrivono posizione ed orientamento della terna 0 rispetto alla terna base, e della terna utensile rispetto alla terna n

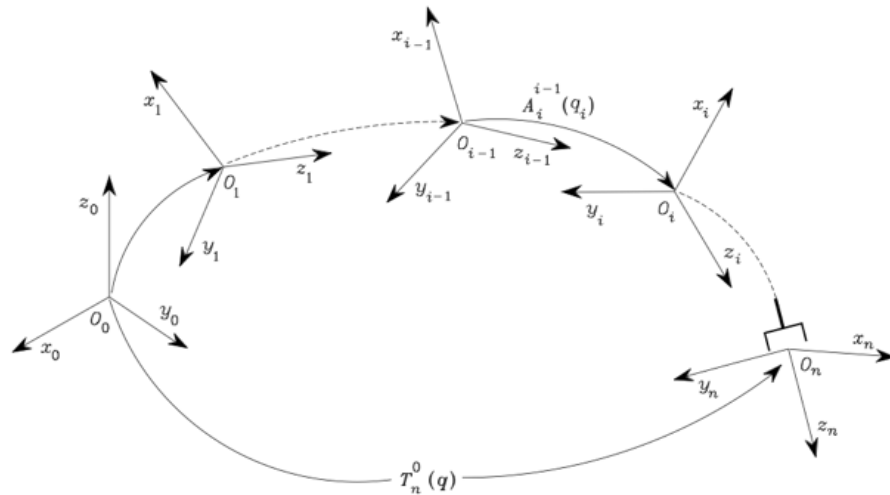


Figura 6 - Trasformazioni omogenee di una catena cinematica aperta

2.5.2. Convenzione di Denavit-Hartenberg (DH)

La convenzione di Denavit-Hartenberg (DH) è un metodo convenzionale utilizzato per descrivere la cinematica dei robot manipolatori. Questa convenzione fornisce una procedura standardizzata per assegnare un sistema di coordinate a ciascun giunto di un manipolatore, consentendo così di rappresentare la trasformazione tra i sistemi di riferimento dei giunti adiacenti, a partire dalle terne solidari a ciascun braccio.

Indicando con i l'asse del giunto che unisce il braccio $i-1$ al braccio i , per definire la terna i usiamo la convenzione di *Denavit-Hartenberg*, o convenzione DH, composta dalle seguenti 4 regole:

- come asse z_i si sceglie quella che giace lungo l'asse del giunto $i+1$;
- l'origine O_i è individuata dall'intersezione dell'asse z_i con la normale comune agli assi z_{i-1} e z_i , e con $O_{i'}$ indichiamo l'intersezione della normale comune con z_{i-1} ;
- l'asse x_i è scelto in modo che sia normale sia all'asse z_{i-1} , che all'asse z_i , con verso definito positivo dal giunto i al giunto $i+1$;
- si sceglie y_i in modo da formare una terna levogira con gli assi x_i e z_i .

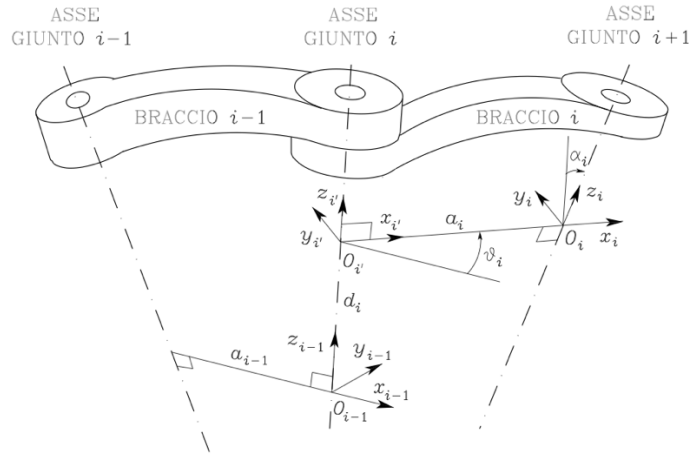


Figura 7 – Convenzione di Denavit-Hartenberg

Le trasformazioni DH vengono quindi utilizzate per descrivere la posa di ciascun braccio del manipolatore rispetto al braccio precedente, cioè determinare posizione ed orientamento della terna i rispetto alla terna $i-1$, tramite l'utilizzo di 4 parametri:

- a_i , che esprime la distanza di O_i da $O_{i'}$,
- d_i , che indica la coordinata su z_{i-1} di $O_{i'}$,
- α_i , che individua l'angolo attorno all'asse x_i tra l'asse z_{i-1} e z_i (considerato positivo in senso antiorario),
- θ_i , che individua l'angolo attorno all'asse z_{i-1} tra l'asse x_{i-1} e x_i (considerato positivo in senso antiorario),

dove a_i e α_i dipendono solo dalla geometria di connessione dei giunti e, a seconda che il giunto in questione sia *rotoidale* o *prismatico*, utilizzeremo rispettivamente θ_i oppure d_i per la descrizione della variabile di giunto.

Da qui si ricava la trasformazione di coordinate che lega la terna i alla terna $i-1$:

$$A_i^{i-1}(q_i) = A_{i'}^{i-1} A_i^{i'} = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i) \cos(\alpha_i) & \sin(\theta_i) \sin(\alpha_i) & a_i \cos(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i) \cos(\alpha_i) & -\cos(\theta_i) \sin(\alpha_i) & a_i \sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.15)$$

2.5.3. Spazio di lavoro

Nella descrizione della posizione di un manipolatore, quando si tratta di descrivere l'orientamento, è conveniente utilizzare una rappresentazione minima in termini delle variabili che descrivono la rotazione della terna utensile rispetto alla terna base, come ad esempio gli angoli di Eulero, che servono a descrivere l'orientamento di un corpo rigido nello spazio tridimensionale rispetto a un sistema di coordinate fisso.

Per individuare la postura di un manipolatore, possiamo utilizzare un vettore, che rappresenta la posizione e l'orientazione dell'end-effector del robot, di dimensione $(m \times 1)$, con $m \leq 6$:

$$x_e = \begin{bmatrix} p_e \\ \phi_e \end{bmatrix} \quad (2.16)$$

con p_e che indica la posizione dell'end-effector e ϕ_e il suo orientamento. Tale vettore definisce lo *spazio operativo*.

D'altra parte, lo *spazio dei giunti* (o delle configurazioni) si riferisce allo spazio in cui "vive" il robot. In questo spazio, ci sono i gradi di libertà del manipolatore, che corrispondono ai movimenti possibili dei suoi giunti. In pratica, questo spazio rappresenta tutte le configurazioni interne del manipolatore che possono essere raggiunte tramite i suoi giunti, ed è definito dal vettore $(n \times 1)$:

$$q = \begin{bmatrix} q_1 \\ \vdots \\ q_n \end{bmatrix} \quad (2.17)$$

possiamo così scrivere l'equazione cinematica (2.12) in tal modo:

$$x_e = k(q) \quad (2.18)$$

Una volta definito lo spazio operativo e lo spazio dei giunti, possiamo definire lo *spazio di lavoro* del manipolatore come la regione descritta dall'origine della terna utensile quando tutti i giunti del manipolatore eseguono tutti i moti possibili. Questo rappresenta la massima estensione spaziale in cui l'end-effector può operare.

Lo *spazio di lavoro raggiungibile* rappresenta la regione in cui l'origine della terna utensile può raggiungere almeno un orientamento. In altre parole, è la parte dello spazio di lavoro in cui il robot può effettivamente raggiungere una posizione.

Lo *spazio di lavoro destro*, invece, è una porzione ancora più limitata dello spazio di lavoro raggiungibile. Rappresenta la regione in cui l'origine della terna utensile può giungere a diversi orientamenti. In questo caso, stiamo considerando non solo le posizioni raggiungibili, ma anche

le varie orientazioni che l'end-effector può assumere. Questo spazio è un sottoinsieme dello spazio di lavoro raggiungibile.

3. Inversione della cinematica differenziale e ridondanza

Come sappiamo l'equazione cinematica diretta è uno strumento che permette di stabilire relazioni funzionali tra le variabili dei giunti di un robot e la posizione del suo organo terminale. Il problema cinematico inverso, invece, riguarda la determinazione delle posizioni dei giunti necessarie per raggiungere una posizione specifica dell'organo terminale. Risolvere questo problema è cruciale per tradurre le specifiche di movimento, espresse nello spazio operativo rispetto all'organo terminale, nei movimenti corrispondenti nello spazio dei giunti.

Tuttavia, il problema cinematico inverso è più complesso per diverse ragioni. In primo luogo, le equazioni coinvolte sono generalmente non lineari, il che rende difficile trovare soluzioni analitiche. In secondo luogo, può esserci più di una soluzione possibile, e in alcuni casi, come nel caso di un manipolatore ridondante, ci possono essere addirittura un numero infinito di soluzioni. Infine, in certi scenari, potrebbe non esistere alcuna soluzione che soddisfi i vincoli del problema.

Un manipolatore, infatti, viene detto *ridondante* quando il suo numero di gradi di libertà è maggiore del numero di variabili necessarie alla caratterizzazione del compito prestabilito, cioè si hanno più possibilità di configurazione rispetto a quelle richieste per raggiungere un dato obiettivo. Ciò avviene quando la dimensione dello spazio operativo è minore della dimensione dello spazio dei giunti ($m < n$).

3.1. Cinematica differenziale e Jacobiano

La *cinematica differenziale* si occupa della relazione tra le velocità dei giunti di un robot e la velocità del suo end-effector (punta finale). Tali relazioni sono descritte da una matrice chiamata *Jacobiano geometrico* che fornisce una rappresentazione delle relazioni tra le velocità dei giunti e le velocità dell'end-effector in termini della sua posizione e orientamento nello spazio. Se invece la posa dell'organo terminale è espressa mediante una rappresentazione in forma minima nello spazio operativo, è possibile calcolare lo *Jacobiano analitico*, che è definito

come la matrice delle derivate parziali delle coordinate dell'end-effector rispetto alle variabili dei giunti del robot; esso viene calcolato utilizzando l'analisi matematica, applicando le regole di derivazione alle equazioni cinematiche dirette del robot.

Indichiamo con $\dot{p}_e$ e ω_e rispettivamente la velocità lineare e la velocità angolare dell'organo terminale, possiamo esprimere l'*equazione cinematica differenziale* in forma compatta del manipolatore nel seguente modo:

$$v_e = \begin{bmatrix} \dot{p}_e \\ \omega_e \end{bmatrix} = J(q)\dot{q} \quad (3.1)$$

in cui $J(6 \times n)$ è lo Jacobiano geometrico del manipolatore nella forma

$$J = \begin{bmatrix} J_P \\ J_O \end{bmatrix} \quad (3.2)$$

dove riconosciamo la matrice $J_P(3 \times n)$ relativa al contributo delle velocità dei giunti alla velocità lineare $\dot{p}_e$, e la matrice $J_O(3 \times n)$ relativa al contributo delle velocità dei giunti alla velocità angolare ω_e dell'end-effector.

Tornando invece all'equazione cinematica di un manipolatore nella forma (2.18), cioè:

$$x_e = k(q) = \begin{bmatrix} p_e \\ \phi_e \end{bmatrix}$$

differenziando rispetto al tempo otteniamo:

$$\dot{x}_e = \begin{bmatrix} \dot{p}_e \\ \omega_e \end{bmatrix} = \begin{bmatrix} \frac{\partial p_e}{\partial q} \dot{q} \\ \frac{\partial \phi_e}{\partial q} \dot{q} \end{bmatrix} = \begin{bmatrix} J_P(q) \\ J_\phi(q) \end{bmatrix} \dot{q} \quad (3.3)$$

dove $J_A(q) = \begin{bmatrix} J_P(q) \\ J_\phi(q) \end{bmatrix}$, prende il nome di *Jacobiano analitico* del manipolatore.

3.2. Inversione della cinematica differenziale

Come abbiamo già detto l'inversione della cinematica differenziale è un processo che consiste nel determinare le variazioni dei giunti necessarie per ottenere una determinata velocità dell'end-effector, mediante l'utilizzo di una trasformazione lineare tra spazio dei giunti e spazio operativo.

Supponiamo di assegnare all'organo terminale una traiettoria di moto specificando la velocità $v_e(t)$ e le condizioni iniziali per posizione ed orientamento. Lo scopo è quello di determinare una possibile traiettoria per i giunti $(q(t), \dot{q}(t))$ che riproduca la traiettoria assegnata.

Le velocità dei giunti $\dot{q}(t)$ possono essere calcolate come segue (nell'ipotesi che il numero di vincoli r sia uguale al numero di gradi di libertà n):

$$\dot{q} = J^{-1}(q)v_e \quad (3.4)$$

da cui possiamo calcolare le posizioni $q(t)$, conoscendo la postura iniziale del robot $q(0)$:

$$q(t) = \int_0^t \dot{q}(\zeta) d\zeta + q(0) \quad (3.5)$$

Per il tempo discreto, usando la regola di integrazione di Eulero:

$$q(t_{k+1}) = q(t_k) + \dot{q}(t_k) \Delta t \quad (3.6)$$

3.3. Manipolatori ridondanti

Per offrire un'analisi più dettagliata della *ridondanza cinematica* è necessario considerare la cinematica differenziale, per legare il numero n dei gradi di libertà del manipolatore, al numero m di variabili necessarie alla caratterizzazione dello spazio operativo, e al numero r di variabili dello spazio operativo necessarie per specificare il compito.

In tal caso l'equazione cinematica differenziale da considerare può essere scritta come:

$$v_e = J(q)\dot{q} \quad (3.7)$$

dove v_e è da intendersi come il vettore ($r \times 1$) delle velocità dell'end-effector, J come la matrice Jacobiana estratta dallo Jacobiano geometrico di dimensione ($r \times n$), e infine $\dot{q}$, di dimensione ($n \times 1$), è il vettore delle velocità dei giunti. Notiamo che se risulta $r < n$, il manipolatore risulta ridondante ed esistono $(n - r)$ *gradi di libertà ridondanti*.

Se il manipolatore è ridondante, lo Jacobiano che si ottiene però è una matrice rettangolare bassa, ciò vorrà dire che esisterà più di una soluzione per l'equazione (3.6). Per risolvere ciò si riformulerà il problema in termini di ricerca dell'ottimo, assegnando cioè la velocità v_e dell'organo terminale e lo Jacobiano J per una data configurazione q , per poi trovare le $\dot{q}$ che soddisfino l'equazione (3.6) e al contempo minimizzino il funzionale di costo quadratico delle velocità:

$$g(\dot{q}) = \frac{1}{2} \dot{q}^T W \dot{q} \quad (3.8)$$

in cui W è una matrice di peso ($n \times n$) simmetrica e definita positiva.

Una soluzione a tale problema la si trova con il *metodo dei moltiplicatori di Lagrange*, che fornisce la soluzione ottima cercata:

$$\dot{q} = W^{-1}J^T(JW^{-1}J^T)^{-1}v_e \quad (3.9)$$

Se la matrice di peso W coincide con la matrice identità I , la soluzione si semplifica come segue:

$$\dot{q} = J^+ v_e \quad (3.10)$$

In cui J^+ è la *pseudo-inversa destra* di J , definita come:

$$J^+ = J^T(JJ^T)^{-1} \quad (3.11)$$

Se però $\dot{q}^*$ è una soluzione dell'equazione lineare (3.6), allora lo sarà anche $\dot{q}^* + P\dot{q}_0$, dove P è il *proiettore del nullo* di J , cioè è un operatore che permette di identificare le componenti delle velocità dei giunti che non influenzano la velocità dell'end-effector, rappresentando le direzioni nel quale il robot non può muoversi nello spazio operativo. Matematicamente il proiettore del nullo di J è definito come:

$$P = (I_n - J^+J) \quad (3.12)$$

A tal punto occorre considerare un nuovo funzionale di costo:

$$g(\dot{q}) = \frac{1}{2}(\dot{q} - \dot{q}_0)^T(\dot{q} - \dot{q}_0) \quad (3.13)$$

il cui obiettivo è minimizzare la norma del vettore $\dot{q} - \dot{q}_0$, ricercando soluzioni $\dot{q}$ che siano più vicine possibili a $\dot{q}_0$.

Procedendo in maniera analoga al caso precedente si ottiene che le soluzioni $\dot{q}$ sono date da:

$$\dot{q} = J^+ v_e + (I_n - J^+J)\dot{q}_0 \quad (3.14)$$

il secondo termine prende il nome di *soluzione omogenea*.

Per quanto riguarda il calcolo di $\dot{q}_0$ una scelta comune è la seguente:

$$\dot{q}_0 = k_0 \left(\frac{\partial \omega(q)}{\partial q} \right)^T \quad (3.15)$$

dove $k_0 > 0$ e $\omega(q)$ è la funzione *obiettivo secondario* delle variabili giunto. Si cerca così di massimizzare localmente tale funzione rispettando anche l'obiettivo primario, cioè il vincolo cinematico.

Ci sono tante possibili scelte della funzione obiettivo, come ad esempio la *distanza dai finecorsa dei giunti*:

$$w(q) = -\frac{1}{2n} \sum_{i=0}^n \left(\frac{q_i - \bar{q}_i}{q_{iM} - q_{im}} \right)^2 \quad (3.16)$$

dove q_{iM} e q_{im} rappresentano rispettivamente la massima e la minima escursione ammessa per la rispettiva variabile di giunto, e $\bar{q}_i$ rappresenta il valore medio della corsa. Così, minimizzando tale distanza, si cerca di portare ogni variabile giunto quanto più vicino al centro corsa.

3.4. Algoritmi per l'inversione cinematica: il CLIK

Nel paragrafo 3.2 si è mostrato come poter calcolare le velocità $\dot{q}$ dei giunti e le posizioni q di quest'ultimi avvalendosi dell'equazione cinematica differenziale. Si deve notare però che l'operazione di integrazione numerica, servita per la ricostruzione delle variabili giunto q , comporta fenomeni di *deriva* della soluzione.

Alle variabili giunto calcolate corrisponde infatti, nello spazio operativo, una posa diversa da quella desiderata dell'end-effector, cioè le $\dot{q}$ calcolate con (3.6) non coincidono con quelle che soddisfano la (3.4): si ricorre quindi ad un algoritmo che tiene conto *dell'errore nello spazio operativo* tra la posa desiderata e quella corrente calcolata dell'organo terminale.

Indichiamo tale errore con:

$$e = x_d - x_e \quad (3.17)$$

e la sua derivata nel tempo

$$\dot{e} = \dot{x}_d - \dot{x}_e \quad (3.18)$$

la quale è possibile scriverla anche nel seguente modo

$$\dot{e} = \dot{x}_d - J_A(q)\dot{q} \quad (3.19)$$

È necessario a tal punto che le $\dot{q}$ abbiano una dipendenza dall'errore e , in modo tale da assicurare una convergenza a zero dell'errore. È opportuno quindi che le variabili giunto q che corrispondono ad una determinata posa dell'organo terminale x_d , vengono ottenute con precisione solo quando l'errore $x_d - k(q)$ rientra in una fascia di tolleranza assegnata.

L'algoritmo di inversione cinematica presentato in tale paragrafo è il CLIK, basato sull'inversa dello Jacobiano

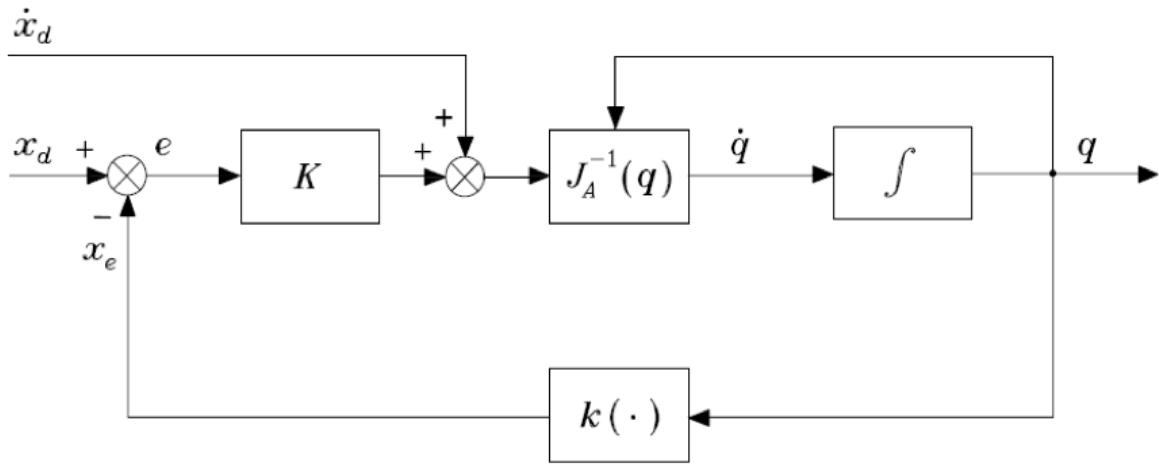


Figura 8 – Schema a blocchi dell'algoritmo CLIK

Ipotizzando che la matrice J_A sia quadrata e non singolare, possiamo scrivere:

$$\dot{q} = J_A^{-1}(q)(\dot{x}_d + Ke) \quad (3.20)$$

la quale porta al seguente sistema lineare equivalente

$$(\dot{e} + Ke) = 0 \quad (3.21)$$

dove K è una matrice definita positiva diagonale.

Il sistema (3.21) quindi risulta asintoticamente stabile con un errore che tende a zero, con una velocità di convergenza che dipende dagli autovalori di K (più sono grandi più la convergenza è veloce).

Nello schema a blocchi osserviamo $k(\cdot)$ che rappresenta la funzione di cinematica diretta (2.18); tale blocco è non lineare, ma è compensato dall'inserimento del blocco $J_A^{-1}(q)$ che rende il sistema lineare.

Inoltre, la presenza dell'integratore nella catena di andata assicura errore a regime nullo per riferimenti costanti ($\dot{x}_d = 0$), mentre l'azione in avanti (*feedforward*) di $\dot{x}_d$ assicura che, in

caso di riferimento variabile nel tempo, l'errore rimanga nullo, in caso di $e(0) = 0$, lungo tutta la traiettoria, per ogni valore del riferimento $\dot{x}_d(t)$ considerato.

Nel caso invece di un manipolatore ridondante, come analizzato precedentemente nel paragrafo 3.3, la (3.20) si scrive

$$\dot{q} = J_A^+(\dot{x}_d + Ke) + (I_n - J_A^+ J_A)\dot{q}_0 \quad (3.22)$$

4. Implementazione dell'algoritmo e simulazione

In tale capitolo si illustrerà ed analizzerà il codice Matlab scritto per l'implementazione dell'algoritmo CLIK, commentando tutti i passaggi e tutte le scelte che ci porteranno a comprendere in che modo è stato possibile controllare il manipolatore robotico Kinova Jaco2 7DOF.

In primo luogo, definisco le variabili temporali in gioco e il numero di punti della traiettoria del robot. Inizializzo, inoltre, l'ascissa curvilinea sulla traiettoria s , la velocità rispetto all'ascissa curvilinea ds , e la velocità di crociera s_cruise , come il doppio della velocità massima del tratto. Considerando che t_{fl} rappresenta la durata del singolo tratto, $\frac{1}{t_{fl}}$ rappresenta la massima velocità istantanea che può essere raggiunta durante il movimento (velocità massima), mentre $\frac{2}{t_{fl}}$ rappresenta il doppio di questa velocità. Quindi, la velocità di crociera è definita come il doppio della velocità massima istantanea del tratto, garantendo che sia maggiore della velocità massima e quindi in grado di mantenere un movimento costante durante la fase di crociera. In questo modo, la velocità di crociera varia tra $\frac{1}{t_{fl}}$ e $\frac{2}{t_{fl}}$, mantenendosi compresa tra questi due valori in relazione alla durata del tratto t_{fl} .

```
tfl = 2; % Durata del singolo tratto
Ts = 0.01; % Tempo di campionamento
t1 = 0:Ts:tfl; % Vettore tempo
N1 = length(t1); % Numero di punti
s = zeros(1,N1); % Ascissa curvilinea
ds = zeros(1,N1);

% definizione vel di crociera compresa tra 1/tfl e 2/tfl
s_cruise=2/tfl;
```

La funzione *trapezoidal*, così definita (*function*[*y*, *dy*, *ddy*] = *trapezoidal*(*y_i*,*y_f*,*dy_c*,*tf*,*t*)), genera l'ascissa curvilinea *s* (dove la lunghezza totale della curva è normalizzata ad 1), salvando i valori di *s(i)* (posizione lungo l'ascissa curvilinea all'istante *i*) e *ds(i)* (velocità lungo l'ascissa curvilinea all'istante *i*) che utilizzerò per calcolarmi le posizioni desiderate dell'organo terminale durante la generazione del doppio quadrato. Ricordiamo che il profilo trapezoidale è usata quando si vuole generare una traiettoria con accelerazione e decelerazione uniformi, per raggiungere una velocità di crociera costante.

```
% ascissa curvilinea
for i = 1:Nl
    [s(i),ds(i),~] = trapezoidal(0,1,s_cruise,tf1,t1(i));
end
```

Calcolo la traiettoria quadrata che eseguirà l'end-effector che consiste in un doppio giro quadrato. Definisco così i punti che formano un quadrato nello spazio tridimensionale: partendo dalla posizione iniziale dell'organo terminale, il primo punto *p0* è semplicemente uguale alla posizione [0; 0; 0]. Successivamente vengono definiti i punti *p1*, *p2*, *p3* e *p4* aggiungendo uno spostamento rispetto al punto precedente lungo gli assi *z* e *y*, e poi lungo gli assi $-z$ e $-y$, rispettivamente, con uno spostamento di 0.15 unità per ciascun asse.

```
% Definizione dei punti per il quadrato
p0 = [0; 0; 0];           % Punto iniziale
p1 = p0 + [0; 0; 0.15];   % Spostamento lungo l'asse z
p2 = p1 + [0; 0.15; 0];   % Spostamento lungo l'asse y
p3 = p2 + [0; 0; -0.15];  % Spostamento lungo l'asse -z
p4 = p3 + [0; -0.15; 0];  % Spostamento lungo l'asse -y
```

Ricavo i vettori *p_desired* e *dp_desired*, rispettivamente per la posizione desiderata e la velocità desiderata, per i quattro lati del quadrato, utilizzando l'ascissa curvilinea, costruendomi il vettore dei tempi, che viene ottenuto concatenando i vettori dei tempi dei quattro tratti

```
% traiettoria di posizione e velocità primo tratto
p1_desired = p0 + (p1-p0)*s;
dp1_desired = (p1-p0)*ds;
t1 = t1;
```

```

% traiettoria di posizione e velocità secondo tratto
p2_desired = p1 + (p2-p1)*s;
dp2_desired = (p2-p1)*ds;
t2 = t1+tfl;

% traiettoria di posizione e velocità terzo tratto
p3_desired = p2 + (p3-p2)*s;
dp3_desired = (p3-p2)*ds;
t3 = t2+tfl;

% traiettoria di posizione e velocità quarto tratto
p4_desired = p3 + (p4-p3)*s;
dp4_desired = (p4-p3)*ds;
t4 = t3+tfl;

% traiettoria complessiva
p_desired = [p1_desired p2_desired p3_desired p4_desired];
dp_desired = [dp1_desired dp2_desired dp3_desired dp4_desired];
t = [t1 t2 t3 t4];

```

Questa parte di codice vista fin ora appartiene alla funzione “*Quadrato.m*”, per la generazione della traiettoria.

Arriviamo quindi alla funzione principale, “*InverseKinematics.m*”, per l’implementazione dell’algoritmo di cinematica inversa

```

clear all
close all;
clc;

% calcolo traiettoria p_desired dp_desired e vettore dei tempi t
Quadrato;
% doppio giro
p_desired = [p_desired p_desired];
dp_desired = [dp_desired dp_desired];
tt = t+4*tfl;
t = [t tt];

N = length(t);           % Numero di punti della simulazione
n = 7;                   % Numero di articolazioni
q = zeros(n, N);         % Posizioni delle articolazioni nel tempo
dq = zeros(n, N);        % Velocità desiderate delle articolazioni nel tempo
dq_0 = zeros(n, N);      % Vettore velocità nello spazio nullo dello Jacobiano

```

```

p_current = zeros(3, N);
error = zeros(3,N);

w = zeros(1, N);
grad_w = zeros(1,n);

```

dove $p_desired$ è il vettore delle posizioni desiderate dell'organo terminale nel tempo (concateno due volte $p_desired$, in modo da ottenere una traiettoria che compie due giri), $dp_desired$ è il vettore delle velocità desiderate dell'organo terminale; tt è il nuovo vettore dei tempi, che corrisponde al vettore dei tempi originale t spostato avanti di quattro volte il tempo originale t_{fl} (concateno poi t con tt , in modo tale da ottenere un vettore dei tempi più lungo che includa entrambi i giri); $error$ è il vettore degli errori tra la posizione desiderata e la posizione attuale dell'organo terminale nel tempo; w è il vettore delle distanze dai fine corsa, inizializzato con zeri per ciascuna articolazione; ed infine $grad_w$ è il vettore dei gradienti di $w(q)$.

Qui definisco il guadagno K utilizzato nell'algoritmo di controllo. Viene utilizzato per regolare l'importanza dell'errore nella cinematica inversa rispetto alla velocità desiderata dell'organo terminale. Un valore più alto di K dà più peso all'errore, di conseguenza, il sistema tenderà a ridurre l'errore più rapidamente, ma potrebbe anche generare oscillazioni indesiderate o instabilità se il guadagno è troppo alto. Tuttavia, un valore più alto di K può essere utile per raggiungere posizioni desiderate più velocemente. Mentre utilizzando un valore più basso di K si darà più peso alla velocità desiderata, riducendo l'importanza dell'errore rispetto alla velocità desiderata dell'organo terminale. Ciò significa che il sistema sarà meno sensibile agli errori, ma potrebbe richiedere più tempo per raggiungere la posizione desiderata. Un valore più basso di K può essere preferibile se si desidera un comportamento più morbido o se si vuole evitare l'instabilità.

k_0 è il valore utilizzato per scalare il contributo delle velocità nello spazio nullo dello Jacobiano (dq_0). È usato per controllare quanto influisce il movimento dei giunti lungo lo spazio nullo del Jacobiano rispetto al movimento desiderato dell'organo terminale. Un valore più alto di k_0 darà più importanza al movimento nello spazio nullo del Jacobiano, mentre un valore più basso darà più importanza al movimento diretto verso la posizione desiderata dell'organo terminale.

```
%definizione guadagno CLIK
K=100;
% Definisco il valore di k0 per il calcolo di dq_0
k0=200;
```

Definisco la matrice DH per descrivere la geometria del manipolatore robotico. Ogni riga della matrice rappresenta i parametri DH di una singola giuntura. I parametri a, α, d, θ altro non sono che le colonne della matrice, ed indicano rispettivamente la lunghezza del link lungo l'asse x (in m), l'angolo tra gli assi z e x misurato intorno all'asse comune (in rad), la distanza tra le origini dei due sistemi di coordinate lungo l'asse z (in m) e l'angolo tra gli assi z dei due giunti consecutivi misurato intorno all'asse comune (in rad). Per tale applicazione, partendo dalla matrice DH del Kinova Jaco2 7DOFs, si è scelto di utilizzare, come configurazione iniziale del braccio, la “*home configuration in Dh convention*”, definita dal vettore (in rad)

$$q_initial(:, 1) = [77; -17; 0; 43; -94; 77; 71] * \pi/180$$

```
% Definizione della matrice DH iniziale (presa dal sito di Cassino)

DH_initial = [
    0, -pi/2, 0.2755, 77*pi/180;
    0, pi/2, 0, -17*pi/180;
    0, pi/2, -0.410, 0;
    0, pi/2, -0.0098, 43*pi/180;
    0, pi/2, -0.3111, -94*pi/180;
    0, pi/2, 0, 77*pi/180;
    0, 0, 0.2638, 71*pi/180
];

% Configurazione iniziale del braccio (in radianti)
%q_initial(:, 1) = [77; -17; 0; 43; -94; 77; 71] * pi/180;

q0 = DH_initial(:,4);
```

Dove q_0 è un vettore contenente i valori iniziali delle posizioni dei giunti, presi dalla quarta colonna della matrice $DH_initial$. In altre parole, q_0 contiene le posizioni iniziali dei giunti del robot.

Calcolo allora la posizione iniziale dell'end-effector utilizzando la funzione “DirectKinematics” (funzione che prende in ingresso la matrice DH); in particolare ricavo la trasformazione omogenea $T_initial$, estraendo la trasformazione omogenea $T_initial_7$ relativa

all'organo terminale (per ottenere la trasformazione dall'organo terminale rispetto alla base del robot, estraiamo l'elemento 7 dall'array `T_initial`). Infine la posizione iniziale dell'organo terminale viene estratta da `T_initial_7`, prendendo le prime tre righe della quarta colonna, che rappresentano le coordinate (x, y, z) della posizione.

```
% Calcolo della posizione iniziale dell'organo terminale utilizzando la
cinematica diretta
T_initial = DirectKinematics(DH_initial);
T_initial_7 = T_initial(:, :, 7); %trasformazione omogenea dell'organo
terminale
p_initial = T_initial_7(1:3, 4); %estraggo la posizione iniziale dell'organo
terminale, considerando solo le prime tre righe dell'ultima colonna (la
quarta) della matrice T_initial_7.
```

Definisco la posizione desiderata rispetto alla posizione iniziale anziché rispetto all'origine del sistema di riferimento, traslo cioè la traiettoria desiderata rispetto alla posizione iniziale, consentendo di specificare `p_desired` in modo relativo rispetto a `p_initial`.

```
% traiettoria a partire dalla posizione iniziale
p_desired = p_initial + p_desired;
```

Definisco i limiti di escursione per ciascuna variabile di giunto, ossia q_{max} e q_{min} aggiungendo la posizione iniziale dei giunti q_0 (è obiettivo infatti minimizzare la distanza dalla posizione iniziale dei primi due giunti del robot), e il valore medio della corsa di ogni giunto q_{mean} . Infine, Inizializzo la variabile `p_current` con la posizione iniziale dell'organo terminale, `p_initial`. Questa linea di codice assegna la posizione iniziale dell'organo terminale all'indice 1 della matrice `p_current`, che rappresenta il momento iniziale della simulazione. Inizializzo poi la matrice DH corrente, `DH_current`, con la matrice `DH_initial`. Questa assegnazione è necessaria all'inizio della simulazione, in quanto la cinematica diretta e inversa del robot utilizzano la matrice DH corrente per calcolare la posizione e l'orientamento dell'organo terminale in base alle posizioni dei giunti. Inizializzo infine il vettore delle posizioni dei giunti `q` con le posizioni iniziali dei giunti. La posizione iniziale dei giunti viene estratta dalla quarta colonna della matrice `DH_initial` (tale assegnazione è importante perché le posizioni dei giunti vengono utilizzate nell'algoritmo di cinematica inversa per calcolare le velocità dei giunti in modo da raggiungere la posizione desiderata dell'organo terminale).

```

% Limiti di escursione dei giunti
q_max = [180; 180; 180; 180; 180; 180; 180] * pi / 180 + q0;
q_min = [-180; -180; -180; -180; -180; -180; -180] * pi / 180 + q0;

% Calcolo del valore medio della corsa per ciascuna variabile di giunto
q_mean = (q_max + q_min) / 2;

%Inizializzazione p_current, DH_current, q
p_current(:, 1) = p_initial;
DH_current = DH_initial;
q(:,1) = DH_initial(:,4);

```

Inizia qui il ciclo for principale con ridondanza, all'interno del quale verrà calcolata la cinematica inversa;

Dapprima inizio definendo la *funzione obiettivo (secondario)* $w(q)$, che ho scelto essere la *distanza dai fine corsa dei giunti* (3.16), per poi calcolare in seguito il gradiente di $w(q)$ rispetto

ai giunti q_i come $\frac{\partial w(q)}{\partial q_j} = -\frac{1}{n} \frac{q_j - \bar{q}_j}{(q_{max,j} - q_{min,j})^2}$

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% ciclo con ridondanza
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

for i = 1:N

    % Calcolo della distanza dai fine corsa w(q)
    for j = 1:2
        w(i) = w(i) + (q(j,i) - q_mean(j))^2 / (q_max(j) - q_min(j))^2;
    end
    w(i) = -w(i) / 2 / n;

    % Calcolo del gradiente di w(q) rispetto a q
    for j = 1:2
        grad_w(j) = -((q(j,i) - q_mean(j)) / (q_max(j) - q_min(j))^2) / n;
    end

    % Calcolo di dq0
    dq_0(:, i) = k0 * (grad_w)';
end

```

infine, calcolo il vettore velocità nello spazio nullo dei giunti dq_0 .

L'errore viene quindi calcolato come la differenza tra la posizione desiderata e quella attuale e memorizzato in `error(:, i)`.

```
% Calcolo dell'errore come differenza tra la posizione desiderata e la
posizione attuale
error(:,i) = p_desired(:,i) - p_current(:,i);
```

Questa parte del codice calcola lo Jacobiano dell'organo terminale utilizzando le posizioni correnti delle articolazioni, rappresentate dalla matrice `DH_current`. Successivamente, viene selezionata solo la parte del Jacobiano relativa alle velocità di traslazione, tramite la selezione delle prime tre righe della matrice Jacobiana completa.

```
% Implementazione dell'algoritmo di cinematica inversa
J = Jacobian(DH_current); % Calcolo del Jacobiano per l'organo terminale

% Considero solo le velocità di traslazione
J_trans = J(1:3, :);
```

Si arriva quindi alla parte centrale del ciclo `for`, il calcolo dell'algoritmo di cinematica inversa, ottenendo `dq` e `q`.

viene calcolata quindi la parte proporzionale della velocità delle articolazioni, mediante l'uso della pseudo-inversa del Jacobiano traslazionale `J_trans`, che tiene conto della discrepanza tra la posizione attuale dell'organo terminale e la posizione desiderata, pesata dal guadagno `K`, e poi la parte nello spazio nullo del Jacobiano traslazionale `J_trans`, che tiene conto degli spostamenti aggiuntivi desiderati delle articolazioni, rappresentati da `dq_0`. Viene utilizzata la differenza tra la matrice identità e la pseudo-inversa del Jacobiano traslazionale per proiettare `dq_0` nello spazio nullo.

Questo fornisce la velocità complessiva delle articolazioni necessaria per guidare l'organo terminale verso la sua posizione desiderata, tenendo conto sia degli errori di posizione sia delle modifiche aggiuntive desiderate dalle articolazioni.

```
%Algoritmo di cinematica inversa
pinvJ = pinv(J_trans);
dq(:, i) = pinvJ * (dp_desired(:,i) + K * error(:,i)) +
+ (eye(n) - pinvJ * J_trans) * dq_0(:, i);
```

Per calcolarmi le posizioni dei giunti nel tempo utilizzo il metodo di integrazione di Eulero per calcolare le nuove posizioni delle articolazioni. Si parte dalla posizione corrente $q(:,i)$ e si aggiunge il prodotto tra il tempo di campionamento T_s e la velocità delle articolazioni $dq(:,i)$; questo calcolo fornisce le nuove posizioni delle articolazioni per il passo temporale successivo $i+1$. Le nuove posizioni delle articolazioni $q(:, i+1)$ vengono utilizzate per aggiornare la quarta colonna della matrice $DH_current$ (che rappresenta le posizioni correnti delle articolazioni) ad ogni iterazione. Utilizzando tale matrice aggiornata, calcolo la trasformazione omogenea corrente $T_current$ (la quale rappresenta la posizione e l'orientamento correnti dell'organo terminale), mediante l'utilizzo della funzione "DirectKinematics". Da $T_current$ vengono estratte le prime tre righe della quarta colonna, che rappresentano la posizione corrente dell'end-effector; tale posizione viene memorizzata in $p_current(:, i+1)$ per il passo temporale successivo.

```
% Aggiornamento delle posizioni delle articolazioni,integrazione
if i < N
    q(:, i+1) = q(:, i) + Ts * dq(:, i); % Utilizza Ts * dq_desired del
        passo temporale corrente

    % Calcolo la matrice DH corrente utilizzando le nuove posizioni delle
        articolazioni
    DH_current(:,4) = q(:,i+1);

    % Calcola la trasformazione omogenea del robot utilizzando la matrice
        DH corrente
    T_current = DirectKinematics(DH_current);

    % Estraggo la posizione corrente dell'organo terminale dalla
        trasformazione omogenea
    p_current(:, i+1) = T_current(1:3, 4, 7);

end

end
```

Salvo le variabili con che presentano ridondanza, poiché dopo verranno calcolate nuovamente le variabili dq e q senza ridondanza, per mostrare poi a differenza delle posizioni dei giunti con e senza dq_0

```
% salvataggio variabili con ridondanza
qr = q;
dqr = dq;

p_currentr = p_current;
errorr = error;

wr = w;
```

Qui un nuovo ciclo for con k0=0

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% ciclo senza ridondanza (k0 = 0)
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% inizializzazione p_current, DH_current, q
p_current(:, 1) = p_initial;
DH_current = DH_initial;
q(:,1) = DH_initial(:,4);

w = zeros(1, N);
grad_w = zeros(1,n);

k0 = 0;

for i = 1:N

    % Calcolo della distanza dal centro corsa w(q)
    for j = 1:n
        w(i) = w(i) + (q(j,i) - q_mean(j))^2/(q_max(j) - q_min(j))^2;
    end
    w(i) = -w(i)/2/n;

    % calcolo del gradiente di w(q) rispetto a q
    for j = 1:n
        grad_w(j) = -((q(j,i) - q_mean(j))/(q_max(j) - q_min(j))^2)/n;
    end

    % calcolo di dq0
    dq_0 = k0*grad_w';

    % Calcolo dell'errore come differenza tra la posizione desiderata e la
    posizione attuale
```

```

error(:,i) = p_desired(:,i) - p_current(:,i);

% Implementazione dell'algoritmo di cinematica inversa con guadagno K=2
J = Jacobian(DH_current); % Calcolo del Jacobiano per l'organo terminale

% Considera solo le velocità di traslazione
J_trans = J(1:3, :);

%Algoritmo di cinematica inversa
pinvJ = pinv(J_trans);
dq(:, i) = pinvJ*(dp_desired(:,i) + K*error(:,i)) + (eye(n) -
pinvJ*J_trans)*dq_0;

% Aggiornamento delle posizioni delle articolazioni,integrazione
if i < N
    q(:, i+1) = q(:, i) + Ts * dq(:, i); % Utilizza Ts * dq_desired del
        passo temporale corrente

    % Calcolo la matrice DH corrente utilizzando le nuove posizioni delle
        articolazioni
    DH_current(:,4) = q(:,i+1);

    % Calcolo la trasformazione omogenea del robot utilizzando la matrice
        DH corrente
    T_current = DirectKinematics(DH_current);

    % Estraggo la posizione corrente dell'organo terminale dalla
        trasformazione omogenea
    p_current(:, i+1) = T_current(1:3, 4, 7);

end
end

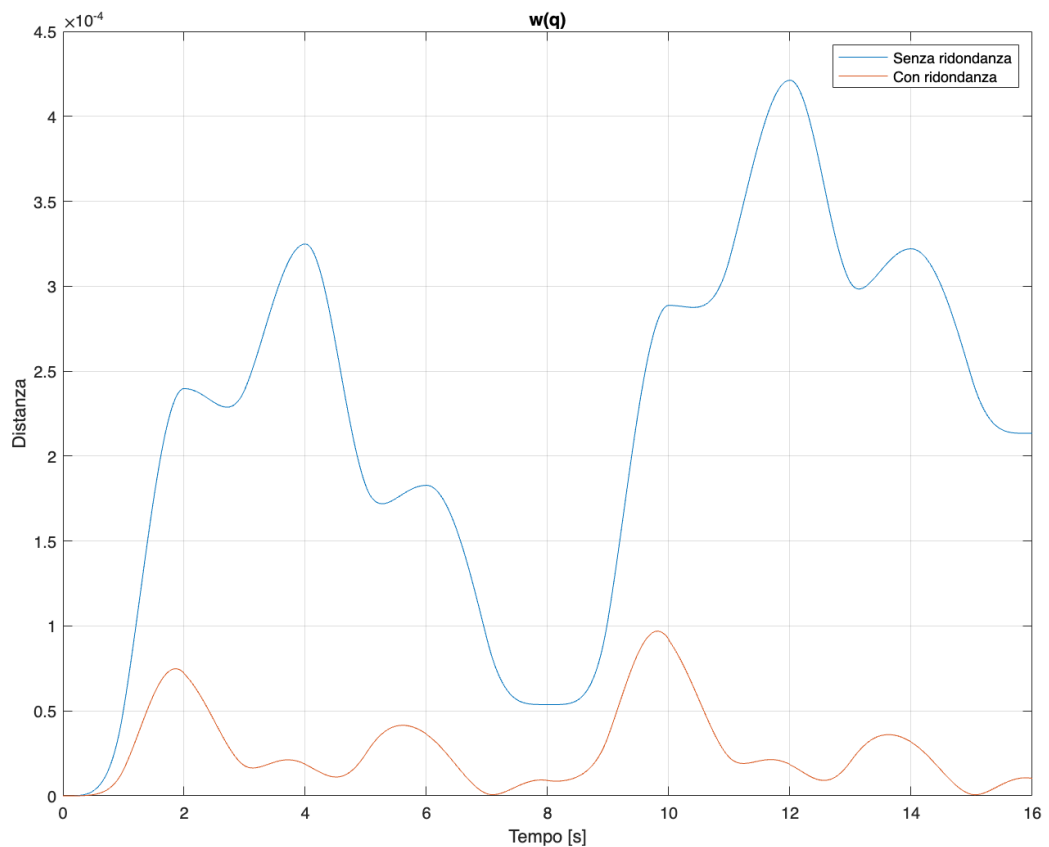
```

Il primo risultato che otteniamo è quello del confronto tra le $w(q)$ ottenute con e senza ridondanza: in particolare notiamo come la presenza di ridondanza consente al sistema di selezionare traiettorie più ottimizzate che minimizzano la distanza dalle posizioni iniziali, rispetto al caso senza ridondanza.

La presenza della ridondanza comporta un calcolo più rapido e più efficiente delle traiettorie dei giunti, per questo il grafico della distanza dalle posizioni iniziali calcolato con ridondanza decresce più rapidamente rispetto al caso senza ridondanza. Questo è considerato un effetto desiderabile, poiché indica una migliore adattabilità e capacità di risposta del sistema alle variazioni nelle condizioni operative. In tal modo, minimizzando la $w(q)$ si fa in modo di

mantenere i giunti del robot il più possibile vicino alle posizioni iniziali (per i primi due giunti), evitando che questi non raggiungono i limiti estremi (si rischierebbe di raggiungere posizioni indesiderabili come singolarità, ecc.)

```
% Confronto tra distanza dalla posizione iniziale con e senza ridondanza
figure('Position', [100, 100, 800, 600]);
plot(t, -w, t, -wr);
legend('Senza ridondanza', 'Con ridondanza');
title('w(q)');
xlabel('Tempo [s]');
ylabel('Distanza');
grid on;
```



Confronto poi le posizioni delle variabili giunto ottenute con e senza ridondanza; notiamo che le posizioni dei primi due giunti, ottenuti con la ridondanza, sono più vicine alle rispettive posizioni iniziali (rispetto a quanto non lo sarebbero senza ridondanza), di seguito riportate:

```
disp(q0);
```

```
1.3439  
-0.2967  
0  
0.7505  
-1.6406  
1.3439  
1.2392
```

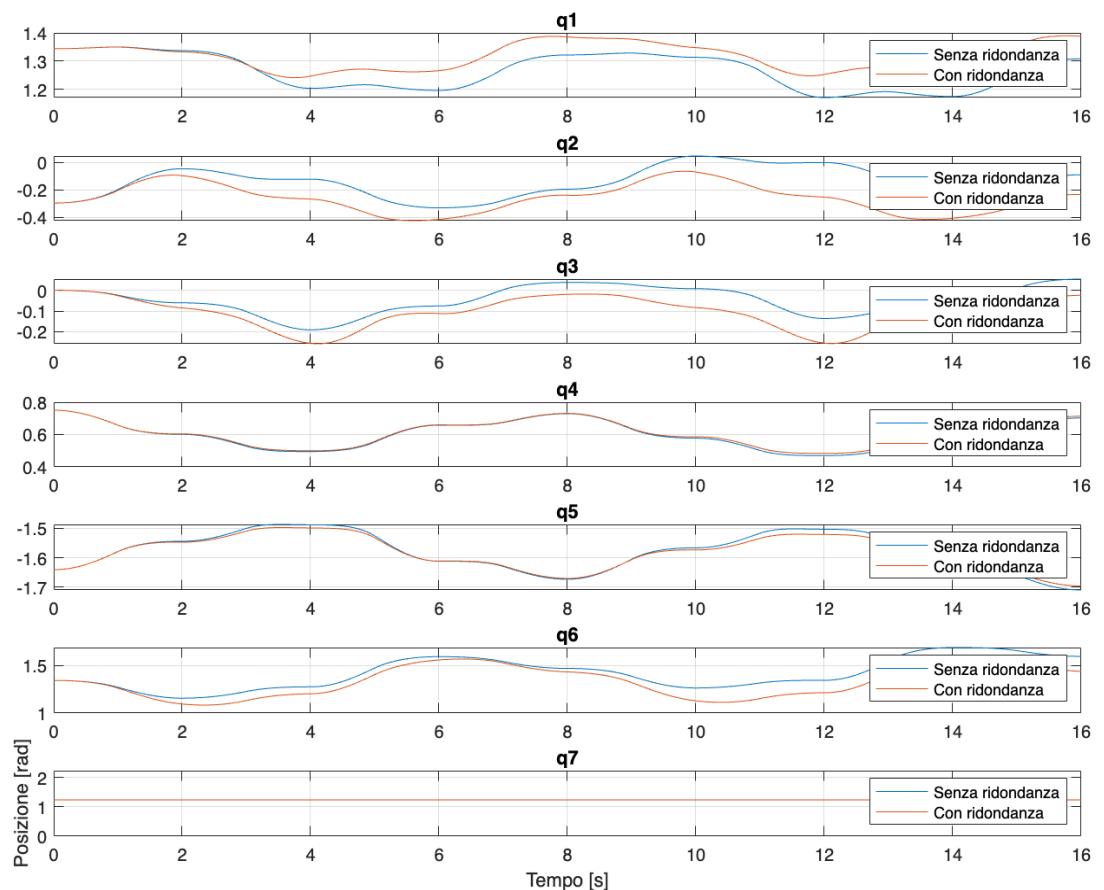
```
% Confronto tra variabili di giunto  
figure('Position', [100, 100, 800, 600]);  
subplot(7,1,1);  
plot(t, q(1,:), t, qr(1,:));  
legend('Senza ridondanza', 'Con ridondanza');  
title('q1');  
grid on;  
  
subplot(7,1,2);  
plot(t, q(2,:), t, qr(2,:));  
legend('Senza ridondanza', 'Con ridondanza');  
title('q2');  
grid on;  
  
subplot(7,1,3);  
plot(t, q(3,:), t, qr(3,:));  
legend('Senza ridondanza', 'Con ridondanza');  
title('q3');  
grid on;  
  
subplot(7,1,4);  
plot(t, q(4,:), t, qr(4,:));  
legend('Senza ridondanza', 'Con ridondanza');  
title('q4');  
grid on;  
  
subplot(7,1,5);  
plot(t, q(5,:), t, qr(5,:));  
legend('Senza ridondanza', 'Con ridondanza');  
title('q5');  
grid on;
```

```

subplot(7,1,6);
plot(t, q(6,:), t, qr(6,:));
legend('Senza ridondanza', 'Con ridondanza');
title('q6');
grid on;

subplot(7,1,7);
plot(t, q(7,:), t, qr(7,:));
legend('Senza ridondanza', 'Con ridondanza');
title('q7');
xlabel('Tempo [s]');
ylabel('Posizione [rad]');
grid on;

```



L'aggiunta di dq_0 porta ad un'ottimizzazione del movimento complessivo, aiutando a garantire che i giunti rimangano all'interno dei loro limiti di escursione durante il movimento, contribuendo così alla stabilità e alla sicurezza del sistema.

Plotto in seguito le velocità dei giunti per ogni istante di tempo

```
% Confronto tra velocità di giunto
figure('Position', [100, 100, 800, 600]);
subplot(7,1,1);
plot(t, dq(1,:), t, dqr(1,:));
legend('Senza ridondanza', 'Con ridondanza');
title('dq1');
grid on;

subplot(7,1,2);
plot(t, dq(2,:), t, dqr(2,:));
legend('Senza ridondanza', 'Con ridondanza');
title('dq2');
grid on;

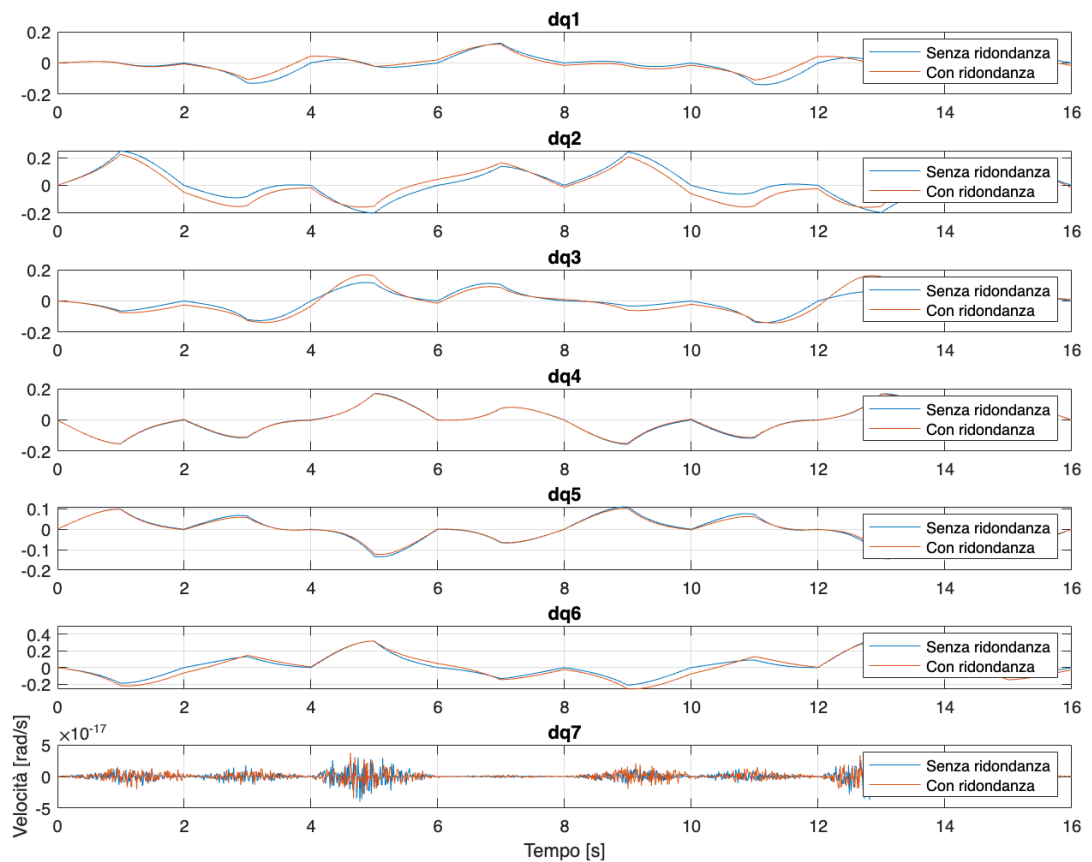
subplot(7,1,3);
plot(t, dq(3,:), t, dqr(3,:));
legend('Senza ridondanza', 'Con ridondanza');
title('dq3');
grid on;

subplot(7,1,4);
plot(t, dq(4,:), t, dqr(4,:));
legend('Senza ridondanza', 'Con ridondanza');
title('dq4');
grid on;

subplot(7,1,5);
plot(t, dq(5,:), t, dqr(5,:));
legend('Senza ridondanza', 'Con ridondanza');
title('dq5');
grid on;

subplot(7,1,6);
plot(t, dq(6,:), t, dqr(6,:));
legend('Senza ridondanza', 'Con ridondanza');
title('dq6');
grid on;

subplot(7,1,7);
plot(t, dq(7,:), t, dqr(7,:));
legend('Senza ridondanza', 'Con ridondanza');
title('dq7');
xlabel('Tempo [s]');
ylabel('Velocità [rad/s]');
grid on;
```

Faccio infine il plot dell'errore con e senza ridondanza

```
% Confronto tra errori di posizione
figure('Position', [100, 100, 800, 600]);
subplot(3,1,1);
plot(t, error(1,:), t, errorr(1,:));
legend('Senza ridondanza', 'Con ridondanza');
title('Errore lungo x');
xlabel('Tempo [s]');
ylabel('Errore [m]');
grid on;

subplot(3,1,2);
plot(t, error(2,:), t, errorr(2,:));
legend('Senza ridondanza', 'Con ridondanza');
title('Errore lungo y');
xlabel('Tempo [s]');
ylabel('Errore [m]');
grid on;

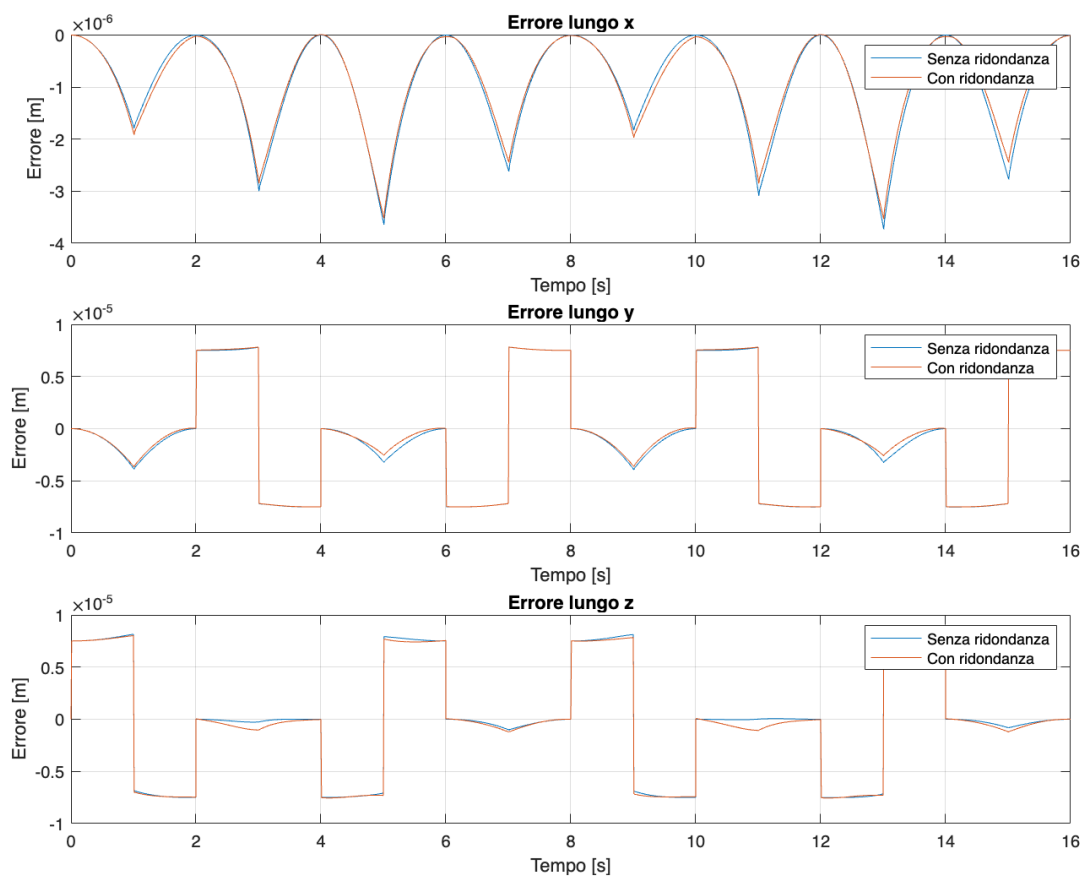
subplot(3,1,3);
```

```

plot(t, error(3,:), t, errorr(3,:));
legend('Senza ridondanza', 'Con ridondanza');
title('Errore lungo z');
xlabel('Tempo [s]');
ylabel('Errore [m]');
grid on;

```

L'andamento dell'errore è causato da variazioni improvvise nella posizione desiderata dell'organo terminale alla fine di ogni traiettoria del lato del quadrato, cioè i “picchi” dell’errore sono dovuti ai cambi di direzione che l’organo terminale effettua alla fine di ogni lato del quadrato.



Di seguito viene riportato il codice intero, il “main_template.m”, che è servito come interfaccia con il software “CoppeliaSim”, che è stato utilizzato per la visualizzazione del Kinova Jaco2 7DOF e per la simulazione dell’algoritmo scritto.

```
% Template to visualize in V-REP the inverse kinematics algorithm developed
%         for the kinova Jaco 2 7-DOF robot
%
% Read Instructions.odt first !
%
% Do not modify any part of this file except the strings within
%   the symbols << >>
%
% G. Antonelli, Introduction to Robotics, 2022

%
% simRemoteApi.start(19999)
%

function q_jaco = main_template(q)
    close all
    clc
    run('InverseKinematics2.m');
    addpath('functions_coppelia/');
    addpath('functions_matlab/');
    porta = 19999;          % default V-REP port

    % decimazione per visualizzazione rapida
    q = downsample(q',4)';

    [n,N] = size(q);        % number of points of the simulation
    q_jaco = zeros(n,N);    % q_jaco(:,i) collects the joint position for
t(i) sent to Jaco
    %q(:,1) = [77  -17 0  43 -94 77 71]'/180*pi; % approximated home
configuration
    q_jaco(:,1) = mask_q_DH2Jaco(q(:,1));

    clc
    fprintf('-----');
    fprintf('\n simulation started ');
    fprintf('\n trying to connect...\n');
    [clientID, vrep ] = StartVrep(porta);

    handle_joint = my_get_handle_Joint(vrep,clientID);    % handle to the
joints
```

```

        my_set_joint_target_position(vrep, clientID, handle_joint, q_jaco(:,1));
% first move to q0
        q_act = my_get_joint_target_position(clientID,vrep,handle_joint,n);% get
the actual joints angles from v-rep

        % main simulation loop
        for i=1:N
            % DH -> Kinova conversion
            q_jaco(:,i) = mask_q_DH2Jaco(q(:,i));
            my_set_joint_target_position(vrep, clientID, handle_joint,
q_jaco(:,i));
            % required only for tempification
            q_act = my_get_joint_target_position(clientID,vrep,handle_joint,n);%
get the actual joints angles from v-rep
            % Kinova -> DH conversion
            q_jaco(:,i) = mask_q_Jaco2DH(q_act);
        end

        DeleteVrep(clientID, vrep);

    end

% constructor
function [clientID, vrep ] = StartVrep(porta)

    vrep = remApi('remoteApi'); % using the prototype file
(remoteApiProto.m)
    vrep.simxFinish(-1); % just in case, close all opened connections
    clientID = vrep.simxStart('127.0.0.1',porta,true,true,5000,5);% start the
simulation

    if (clientID>-1)
        disp('remote API server connected successfully');
    else
        disp('failed connecting to remote API server');
        DeleteVrep(clientID, vrep); %call the destructor!
    end

    % to change the simulation step time use this command below, a custom dt
in v-rep must be selected,
    % and run matlab before v-rep otherwise it will not be changed
    % vrep.simxSetFloatingParameter(clientID,
vrep.sim_floatparam_simulation_time_step, 0.002,
vrep.simx_opmode_oneshot_wait);
    vrep.simxStartSimulation(clientID, vrep.simx_opmode_oneshot);
end

% destructor
function DeleteVrep(clientID, vrep)

```

```

        vrep.simxPauseSimulation(clientID,vrep.simx_opmode_one-shot_wait); % pause
simulation
        %vrep.simxStopSimulation(clientID,vrep.simx_opmode_one-shot_wait); % stop
simulation
        vrep.simxFinish(clientID); % close the line if still open
        vrep.delete(); % call the destructor!
        disp('simulation ended');

    end

function my_set_joint_target_position(vrep, clientID, handle_joint, q)

    [m,n] = size(q);
    for i=1:n
        for j=1:m
            err =
vrep.simxSetJointPosition(clientID,handle_joint(j),q(j,i),vrep.simx_opmode_one
shot);
                if (err ~= vrep.simx_error_noerror)
                    fprintf('failed to send joint angle q %d \n',j);
                end
            end
        end
    end

end

function handle_joint = my_get_handle_Joint(vrep,clientID)

    [~,handle_joint(1)] =
vrep.simxGetObjectHandle(clientID,'Revolute_joint_1',vrep.simx_opmode_one-shot_
wait);
    [~,handle_joint(2)] =
vrep.simxGetObjectHandle(clientID,'Revolute_joint_2',vrep.simx_opmode_one-shot_
wait);
    [~,handle_joint(3)] =
vrep.simxGetObjectHandle(clientID,'Revolute_joint_3',vrep.simx_opmode_one-shot_
wait);
    [~,handle_joint(4)] =
vrep.simxGetObjectHandle(clientID,'Revolute_joint_4',vrep.simx_opmode_one-shot_
wait);
    [~,handle_joint(5)] =
vrep.simxGetObjectHandle(clientID,'Revolute_joint_5',vrep.simx_opmode_one-shot_
wait);
    [~,handle_joint(6)] =
vrep.simxGetObjectHandle(clientID,'Revolute_joint_6',vrep.simx_opmode_one-shot_
wait);
    [~,handle_joint(7)] =
vrep.simxGetObjectHandle(clientID,'Revolute_joint_7',vrep.simx_opmode_one-shot_
wait);

```

```

end

function my_set_joint_signal_position(vrep, clientID, q)

    [~,n] = size(q);

    for i=1:n
        joints_positions = vrep.simxPackFloats(q(:,i)');

[err]=vrep.simxSetStringSignal(clientID,'jointsAngles',joints_positions,vrep.s
imx_opmode_oneshot_wait);

        if (err~=vrep.simx_return_ok)
            fprintf('failed to send the string signal of iteration %d \n',i);
        end
    end
    pause(8);% wait till the script receives all data, increase it if dt is
too small or tf is too high

end

function angle = my_get_joint_target_position(clientID,vrep,handle_joint,n)

    for j=1:n

vrep.simxGetJointPosition(clientID,handle_joint(j),vrep.simx_opmode_streaming)
;
        end

        pause(0.05);

        for j=1:n

[err(j),angle(j)]=vrep.simxGetJointPosition(clientID,handle_joint(j),vrep.simx
_opmode_buffer);
            end

            if (err(j)~=vrep.simx_return_ok)
                fprintf(' failed to get position of joint %d \n',j);
            end
        end

    end
end

```

Tutte le funzioni (compreso il main_template) utilizzate all'interno dell'algoritmo sono state prese dal sito lairobotics@unicas.it.

Bibliografia

Siciliano B., Sciavicco L., Villani L., Oriolo G. Robotica, Modellistica, pianificazione e controllo. McGraw-Hill, Terza edizione.

Sitografia

kinovarobotics.com

docenti.unina.it

lairobotics@unicas.it

rocco.faculty.polimi.it

Ringraziamenti

Grazie anzitutto al Professore Luigi Villani per la sua incredibile disponibilità ed attenzione durante il mio percorso di tesi, dimostrandosi uno dei professori più presenti ed attenti che io abbia mai incontrato.

Un grazie speciale va ai miei amici: ai miei due gemellini preferiti Gianluca e Pierluigi, la scoperta più bella che potessi fare in questi ultimi anni, senza di loro sarebbe tutto più noioso; a Riccardo e Ludovica, senza i quali non riuscirei a stare, e verso i quali ho sviluppato ormai una dipendenza; ad Amedeo ed alla sua risata che mi migliora le serate; a Lorenzo, la mia spalla in questi anni su cui ho sempre potuto contare. Vi voglio bene.

Grazie anche ai miei fratelli in terra sarda Nicolò e Tommaso, la cui sola distanza ci divide. Vi sono grato per tutti i ricordi vissuti assieme.

Ai miei amici di una vita, Ivan ed Alessandro, grazie ai quali ho scoperto il vero significato della parola amicizia.

Al mio secondo fratello maggiore, Marco, il mio mentore calcistico e non solo.

Grazie ai miei zii e ai miei cugini, ma soprattutto a chi mi ha accompagnato nel mio primo esame e nel mio ultimo. C'è anche il tuo contributo in questo traguardo, grazie mille Isa.

Alle mie adorato nonne e ai miei cari nonni, che mi hanno insegnato i valori della famiglia, dell'affetto e dell'amore, nonché della dedizione al duro lavoro che viene sempre ripagato. Siete per me un esempio da seguire.

Tutto ciò però è dedicato alle persone più importanti per me, alla mia meravigliosa Mamma, al mio splendido Papà, ed anche a Kikka. A voi devo ogni cosa, siete tutto.

Ed infine al mio fratellone Armando, il mio faro in mezzo al mare, il mio Virgilio che mi ha fatto sempre da mentore, la mia forza in tutto e per tutto, senza di te sarei perso. Vi amo.

